



Important

- Ensure you have access to **AWS Transform Custom (ATX)** CLI
- Complete the [Setting Up Kiro](#) section
- A **.NET Framework** application ready for modernization (provided in this lab)

Modernize .NET Applications with AWS Transform Custom and Kiro

Introduction

AWS Transform Custom (commonly referred to as **ATX Custom**) is an agentic AI service that performs large-scale modernization of software, code, libraries, and frameworks to reduce technical debt. Combined with **Kiro** (IDE and CLI), it provides a powerful end-to-end workflow for modernizing legacy .NET Framework applications to cross-platform .NET (e.g., .NET 8 or .NET 10).

Modernizing .NET Framework applications is a common challenge for organizations running Windows-based workloads. According to [AWS](#), porting to modern .NET on Linux can reduce operating costs by up to 40% through Windows licensing savings, improve performance by 1.5–2x, and deliver up to 50% better scalability.

What is AWS Transform Custom?

AWS Transform Custom uses agentic AI to handle diverse modernization scenarios including language version upgrades, API and service migrations, framework upgrades, code refactoring, and organization-specific transformations. It offers both a **CLI** (`atx custom`) and a **web interface** to suit different modernization needs.

Key capabilities include:

- **Natural language-driven transformation definition** — Create custom transformations using natural language, documentation, and code samples
- **Transformation execution** — Apply transformations consistently and reliably across multiple codebases
- **Continual learning** — Automatically improve transformation quality from every execution and developer feedback
- **AWS-managed transformations** — Use ready-to-use, AWS-vetted transformations for common scenarios (e.g., `AWS/java-version-upgrade`, `AWS/python-version-upgrade`, `AWS/nodejs-version-upgrade`)

How ATX Custom and Kiro Work Together

The modernization workflow combines both tools in a complementary way:

1. **AWS Transform Custom (ATX)** — Analyzes your .NET Framework codebase, generates a tailored modernization plan, and automatically converts application code, builds the output, runs unit tests, and commits results to a new branch in your repository.
2. **Kiro (IDE/CLI)** — Helps you fix remaining compilation issues, resolve package version conflicts, and iteratively build and test the modernized application using natural language prompts and spec-driven development.

This combination enables what AWS calls "**vibe modernization**" — using AI-powered tools collaboratively to port and modernize .NET applications faster than traditional manual methods.

Why Modernize .NET Framework to Modern .NET?

Benefit	Detail
Cost Savings	Up to 40% reduction in operating costs by removing Windows licensing dependencies
Performance	1.5–2x faster application performance on modern .NET
Scalability	Up to 50% better scalability for growing workloads
Cross-Platform	Run on Linux, containers, and serverless platforms
Long-Term Support	.NET Framework 4.8 is in maintenance mode; modern .NET receives active development

What You'll Learn

Throughout this lab, you'll discover how to:

- Set up and configure the ATX Custom CLI for .NET modernization
- Analyze a legacy .NET Framework application and generate a modernization plan
- Execute an AI-powered transformation to port .NET Framework code to modern .NET
- Use Kiro CLI to fix conversion layout issues, set up databases, and deploy the application
- Verify security and technical debt improvements from the transformation
- Deploy and run the modernized application on Linux using Docker
- Understand the end-to-end workflow of combining ATX Custom with Kiro

By the end of this lab, you'll have hands-on experience modernizing a .NET Framework application using agentic AI tools, and understand how to apply this workflow to your own legacy .NET codebases.

Prerequisites

Before starting this lab, ensure you have:

- Completed the [Setting Up Kiro](#) section
- .NET SDK installed (version 8.0 or later recommended)
- A basic understanding of .NET development concepts
- Familiarity with command-line interfaces



Note

The AWS Transform Custom CLI (`atx`) will be installed and configured as part of this lab (Section 121).

Let's begin! 🚀 ⭐

Previous

Next